CS 180 · PROGRAMMING PROJECT 2

Fun with Filters & Frequencies

Convolutions, edges, hybrid images, and multiresolution blending.

Cool project about frequencies.

PART 1

Fun with Filters

1.1 Convolutions from Scratch numpy only

My code used only numpy: the four for loops use simple indexing, and the two for loops make use of numpy vectorized operations. How it works is it slides the kernel over the image, calculates the sum of the multiplied values, and puts that as the answer. The box filter averages the values in the box, causing a blur; `Dx` looks at the horizontal change, finding vertical edges; `Dy` does a similar thing but finds horizontal edges. For padding I

used `numpy.pad` (it defaults to constant 0 padding), which is how I deal with boundaries, and is the same as scipy's `convolve2d`. In comparison with `scipy.signal.convolve2d`: runtime-wise, the four-loop version is way slower, and the two-loop version is similar in speed (because of the vectorized operations). Scipy's convolve2d is way faster (use of compiled code and probably efficient vectorized operations), often at least 10x faster.

`naiveConvolution — four loops`

```
def naiveConvolution(img,kernel):
    #using same padding
    kh,kw = kernel.shape
    H,W = img.shape
    ans = np.zeros(img.shape)
    paddingW = (kw-1)//2
    paddingH = (kh-1)//2
    img = np.pad(img, ((paddingH, paddingH), (paddingW, paddingW)))


    for y in range(H):
        for x in range(W):
            tempans=0
            for j in range(kh):
                for i in range(kw):
                    tempans+=img[y+j][x+i]*kernel[j][i]
            ans[y,x]=tempans
    ans=np.array(ans)
    return ans
```

`optimizedConvolution — two loops`

```
def optimizedConvolution(img,kernel):
    kh,kw = kernel.shape
    H,W = img.shape
    ans = np.zeros(img.shape)
    paddingW = (kw-1)//2
    paddingH = (kh-1)//2

    img = np.pad(img, ((paddingH, paddingH), (paddingW, paddingW)))

    for y in range(H):
        for x in range(W):
            ans[y,x]=np.sum(img[y:y+kh,x:x+kw]*kernel)
```

```
    ans=np.array(ans)
    return ans
```



INPUT · GRAYSCALE



9×9 BOX FILTER



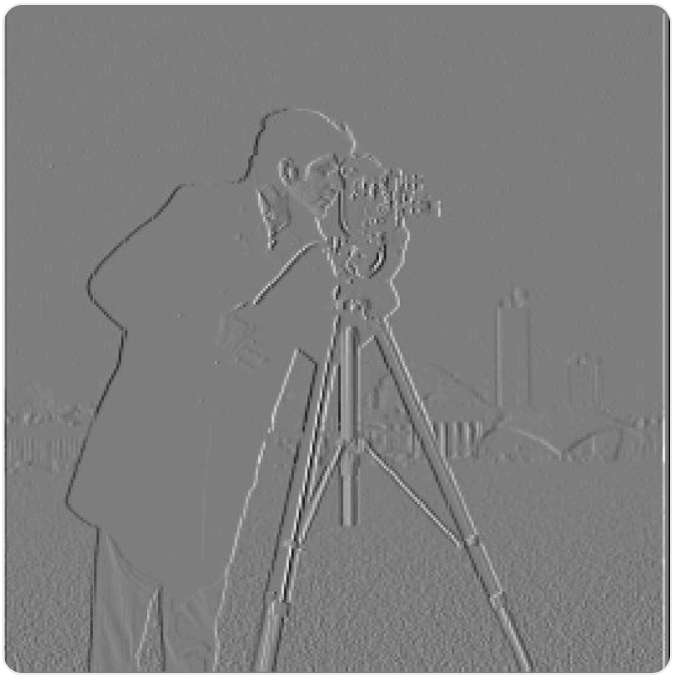
D_X



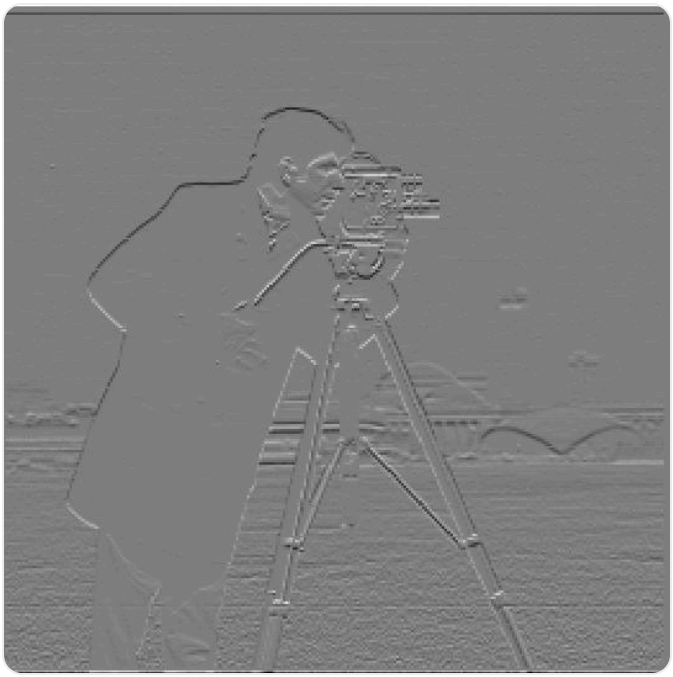
D_Y

1.2 Finite Difference Operator

I convolve with D_x and D_y to get the partial derivatives. I compute the gradient magnitude at each pixel using the formula sqrt(dx^2 + dy^2). This marks where the change in intensity is high, not necessarily the direction. Then I only select those above my threshold. I chose a threshold of 0.2, because even though there was still a little noise at the bottom, it keeps more of the edges of the buildings behind, which are lost at the next threshold.



∂/∂X

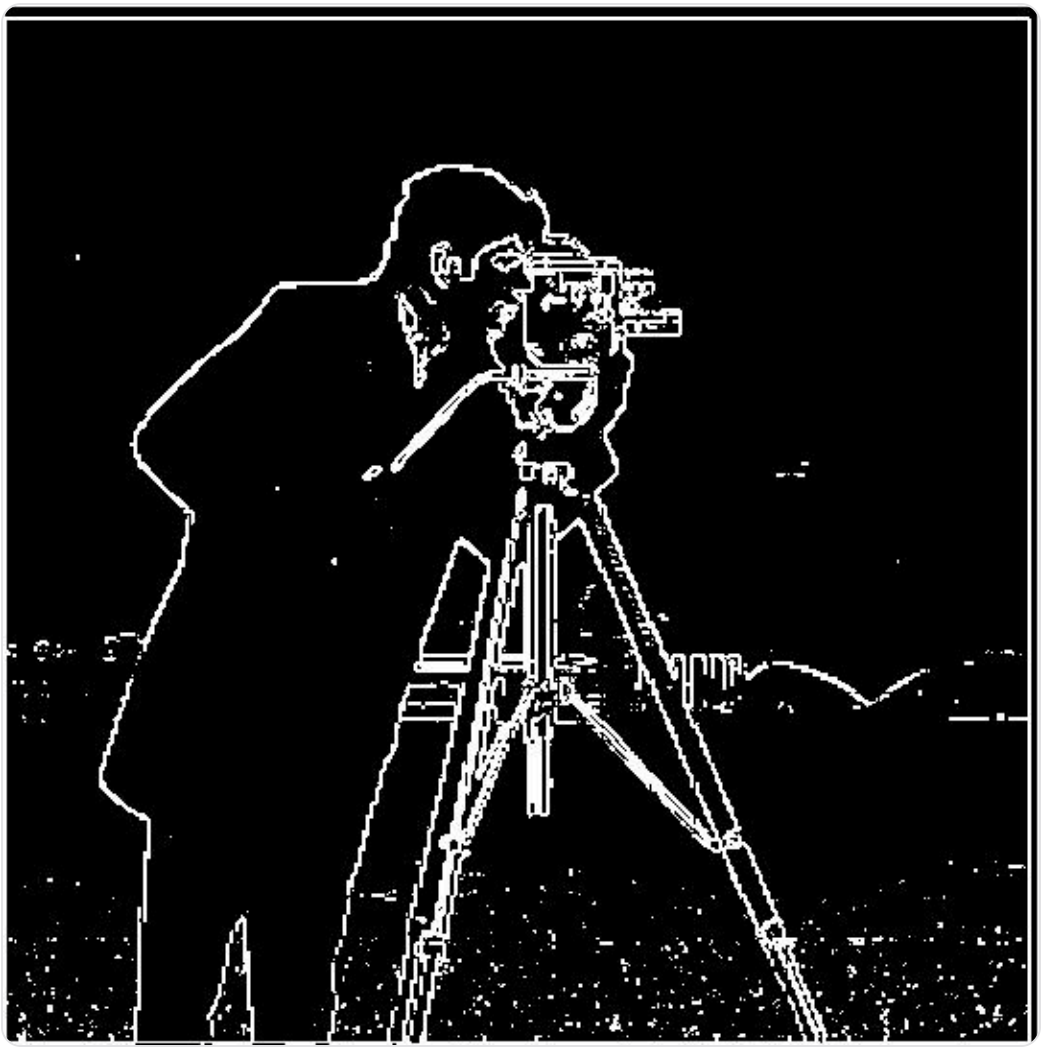


∂/∂Y



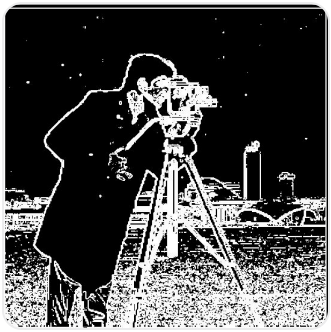
GRADIENT MAGNITUDE

Chosen binarized edge image:

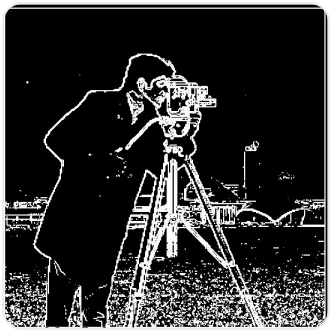


BINARIZED EDGES · CHOSEN THRESHOLD = 0.2

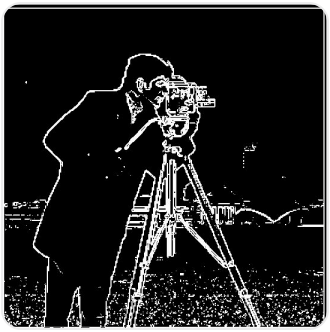
Threshold sweep (used to pick the value):



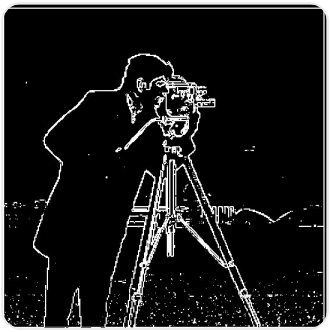
THR 0.05



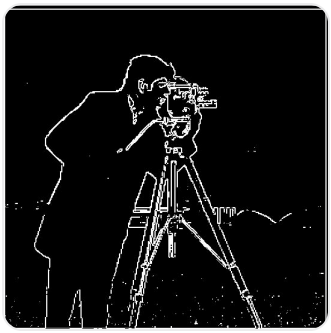
THR 0.1

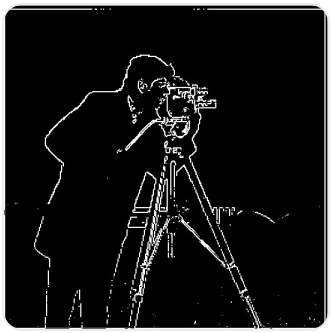


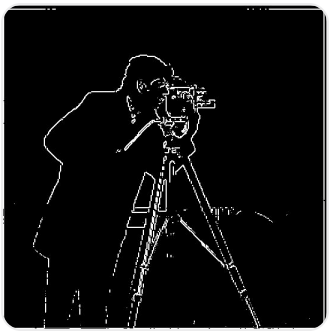
THR 0.15

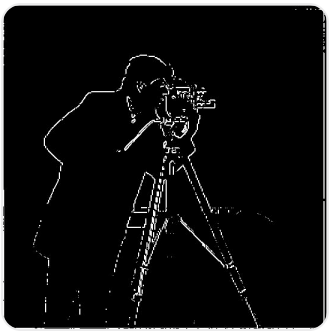


THR 0.2



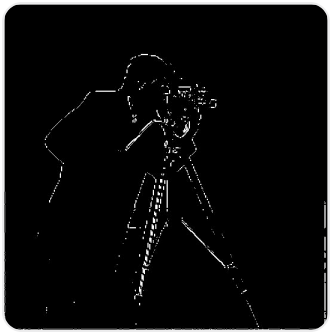






THR 0.25 THR 0.3 THR 0.35 THR 0.4



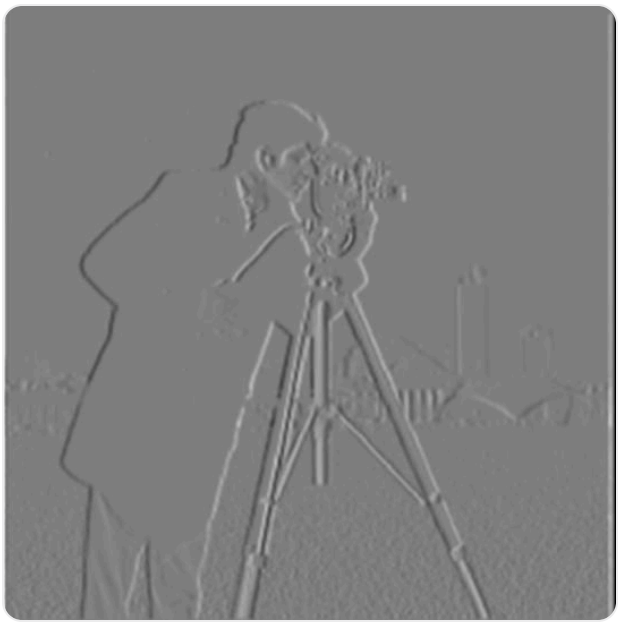


THR 0.45 THR 0.5

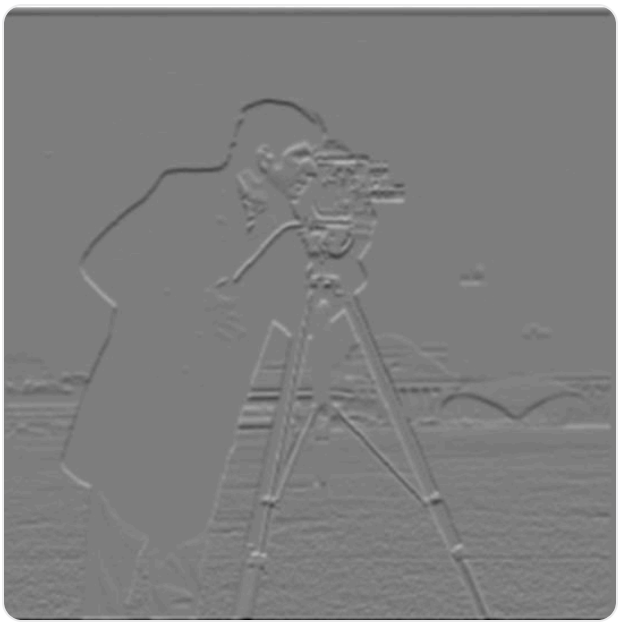
1.3 Derivative of Gaussian (DoG) Filter

I built a 2D gaussian (5×5 kernel, sigma = 1.5 for both approaches, so it is a fair comparison) following the procedure mentioned in the instructions. In one approach I applied the kernel to the image and then dx and dy; in the single convolution I apply the gaussian to each and then convolve. What I notice is that, as you can see by the sweep, the noise gets removed very quickly after differentiating, at lower thresholds. Both the single convolution and the gaussian produce the same image (some nuance with padding, but regardless). This is because of the associativity of the operation, where the order doesn't really matter.

(a) Gaussian smoothing, then finite difference



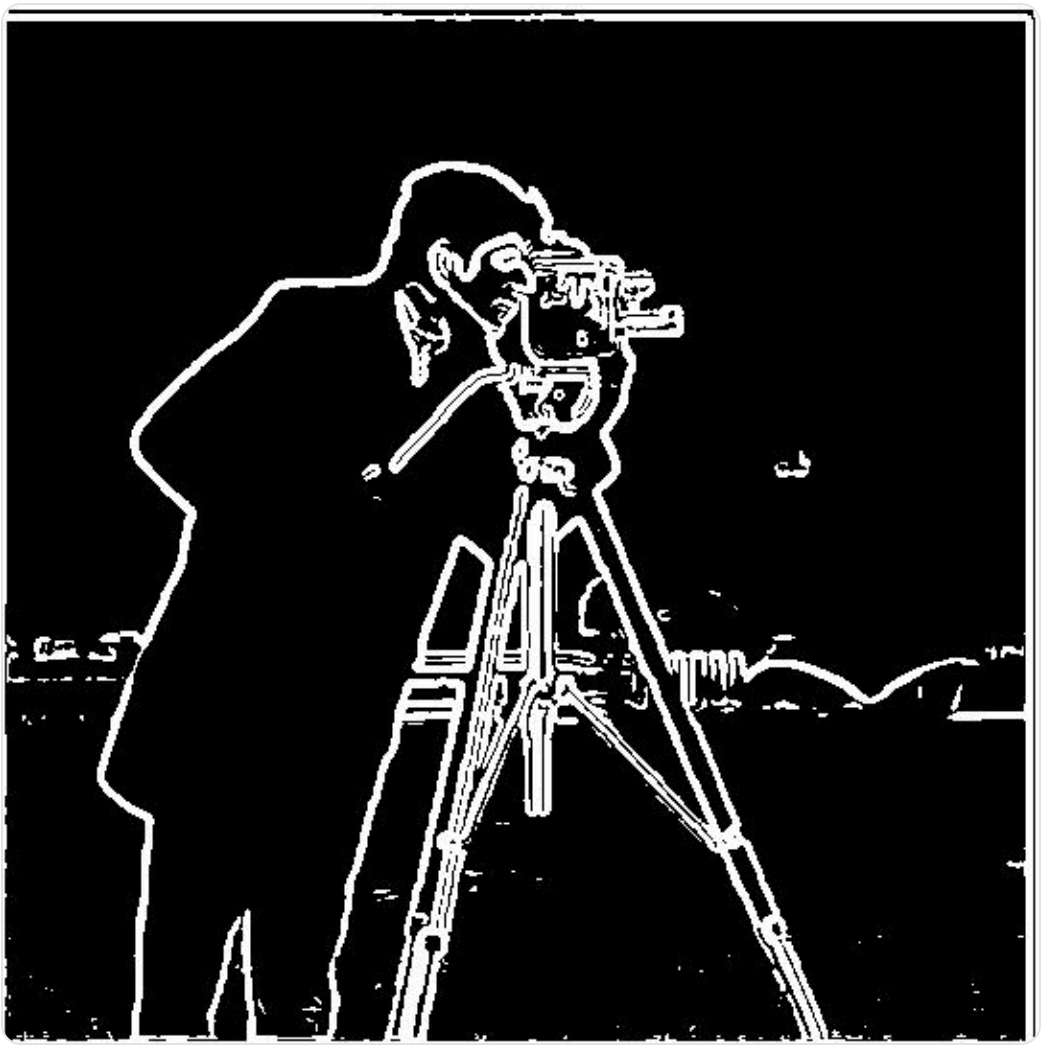
∂/∂X (SMOOTHED)



∂/∂Y (SMOOTHED)

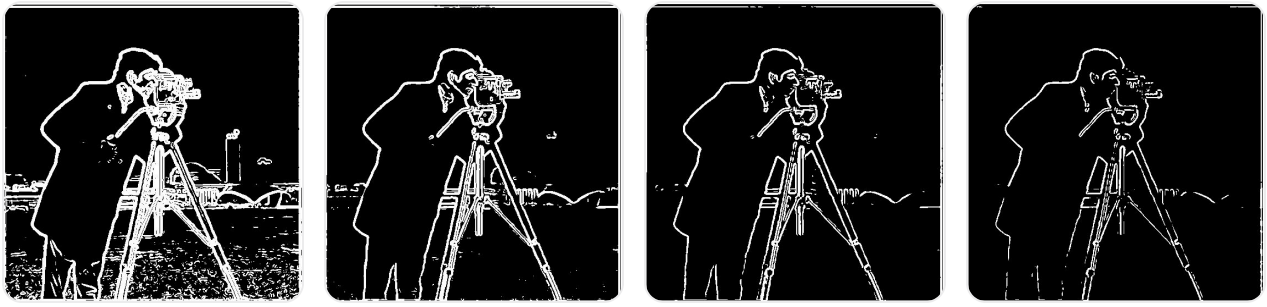


GRADIENT MAGNITUDE

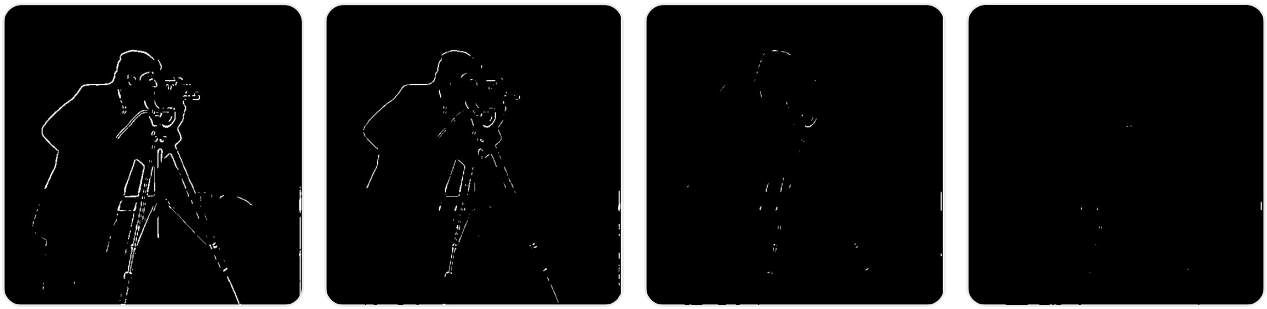


BINARIZED EDGES · CHOSEN THRESHOLD = 0.1

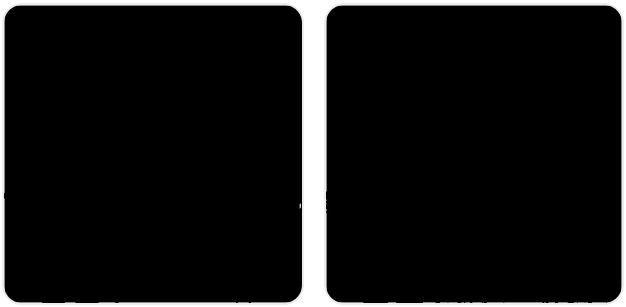
Threshold sweep:



THR 0.05 THR 0.1 THR 0.15 THR 0.2



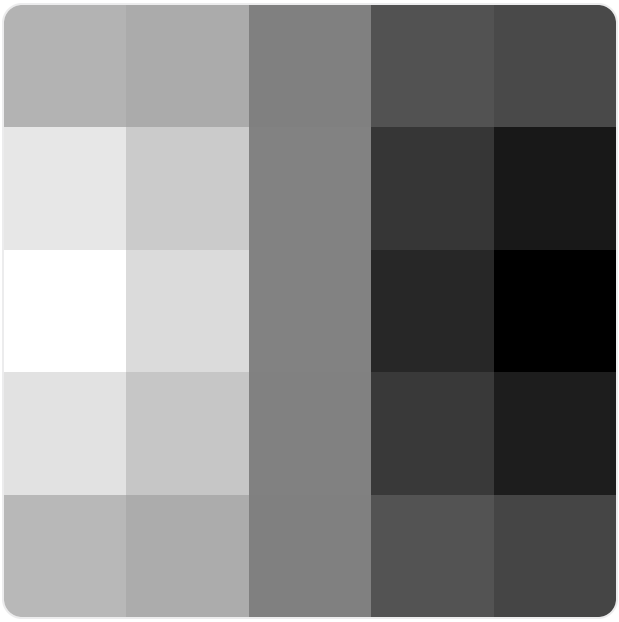
THR 0.25 THR 0.3 THR 0.35 THR 0.4



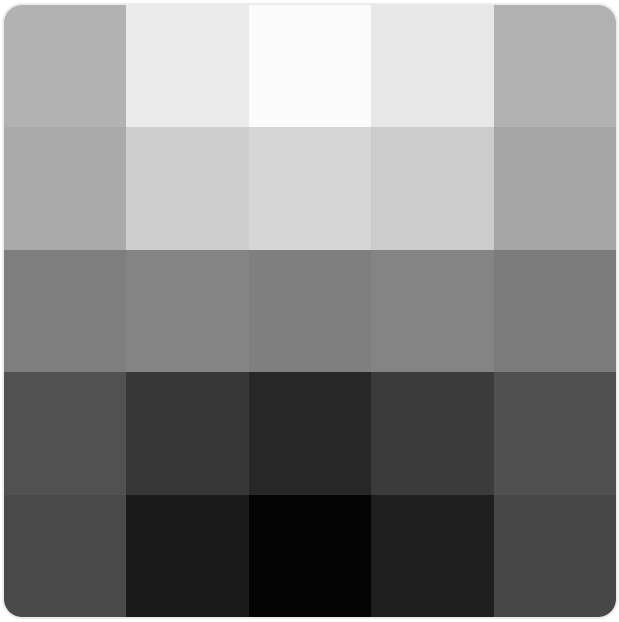
THR 0.45 THR 0.5

(b) Single-convolution Derivative-of-Gaussian

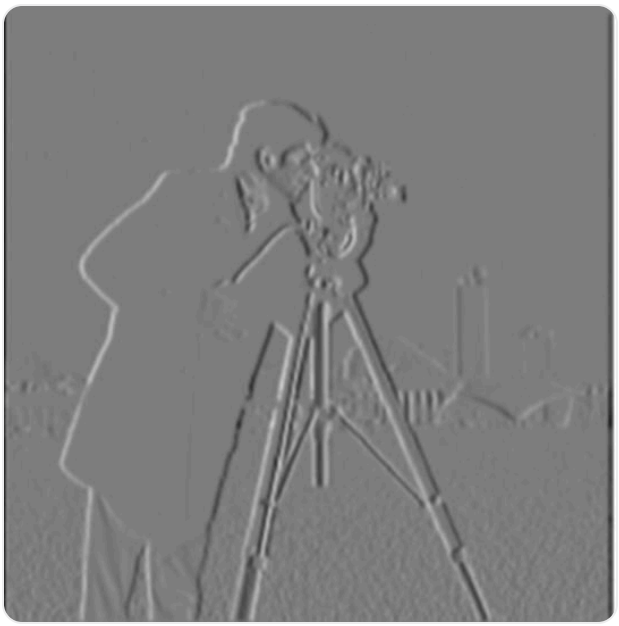
DoG filters, visualized:



DOG FILTER · X



DOG FILTER · Y



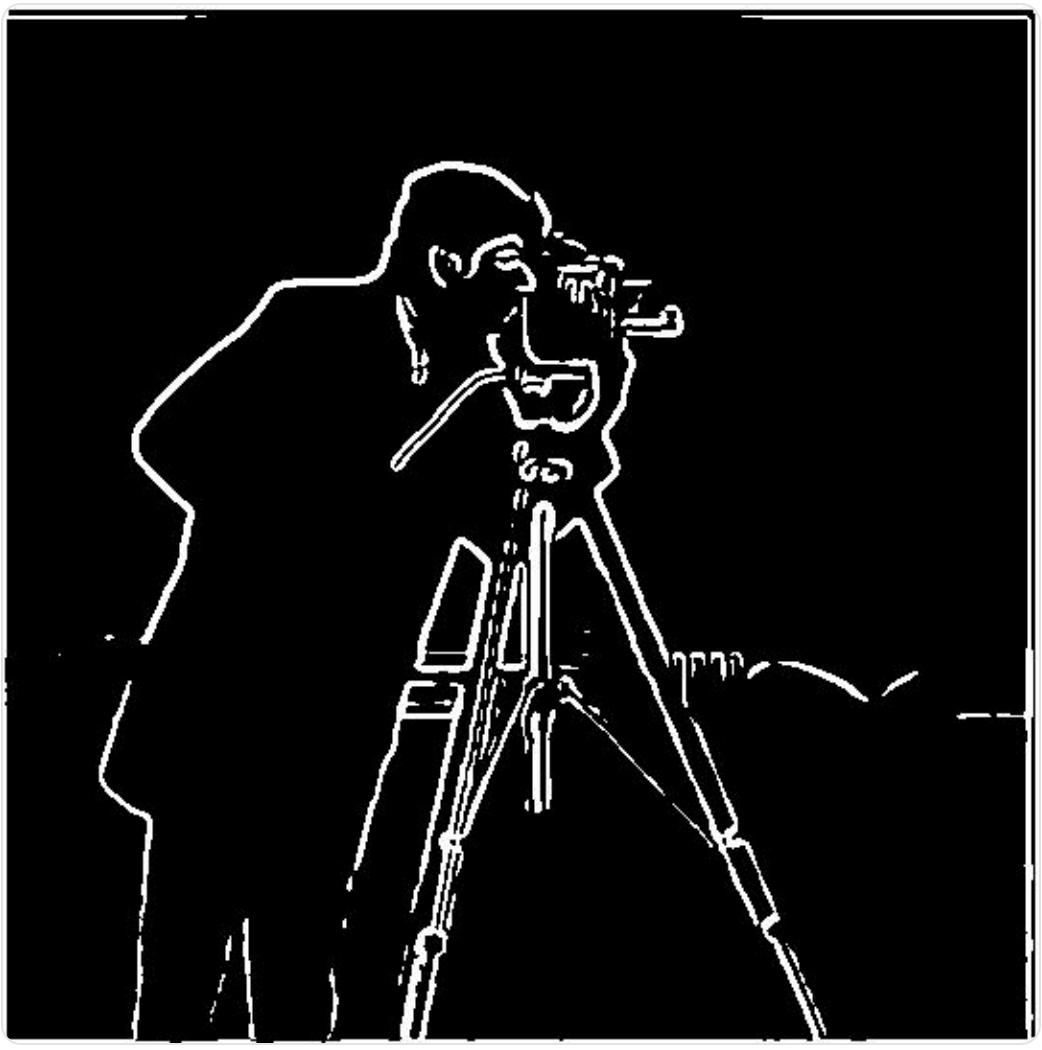
DOG ∂/∂X



DOG ∂/∂Y

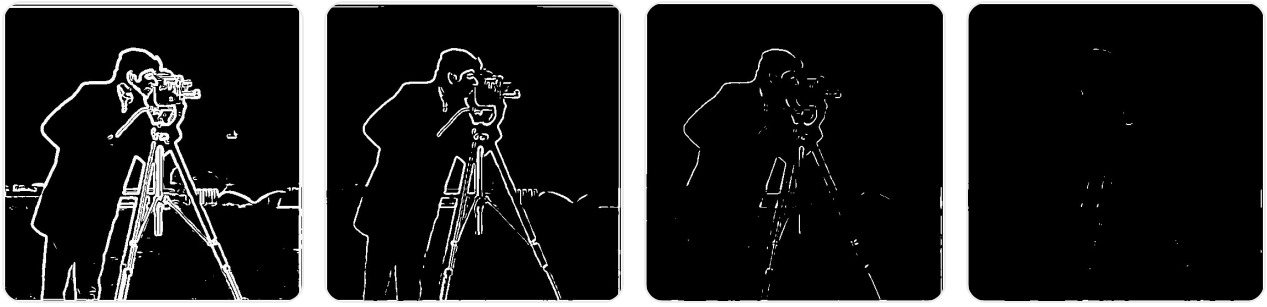


GRADIENT MAGNITUDE

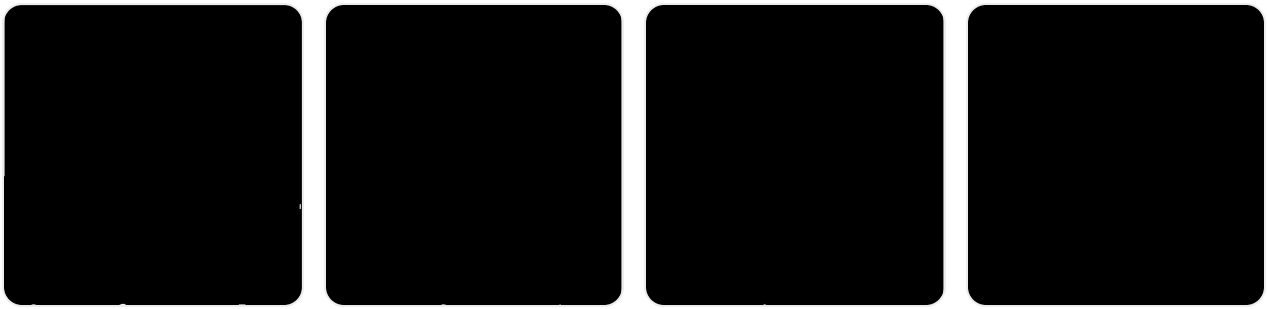


BINARIZED EDGES · CHOSEN THRESHOLD = 0.1

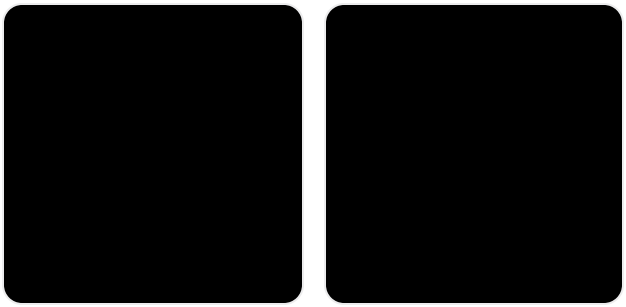
Threshold sweep:



THR 0.05 THR 0.1 THR 0.15 THR 0.2



THR 0.25 THR 0.3 THR 0.35 THR 0.4



THR 0.45 THR 0.5

PART 2

Fun with Frequencies

2.1 Image "Sharpening"

We want to extract the high frequencies of the image (which supposedly correspond to detail). To get this we can blur the image with a gaussian kernel (I chose a 5×5 kernel with sigma = 1). You can then apply this back with alpha. When alpha gets high we see the

features get exaggerated, but as it becomes too high we start to see weird artifacts appear.

Taj Mahal



ORIGINAL



BLURRED (LOW-PASS)



HIGH FREQUENCIES

Sharpening amount sweep:



AMOUNT 1



AMOUNT 2



AMOUNT 3



AMOUNT 4



AMOUNT 5

Evaluation: sharp → blurred → re-sharpened (LeCun)

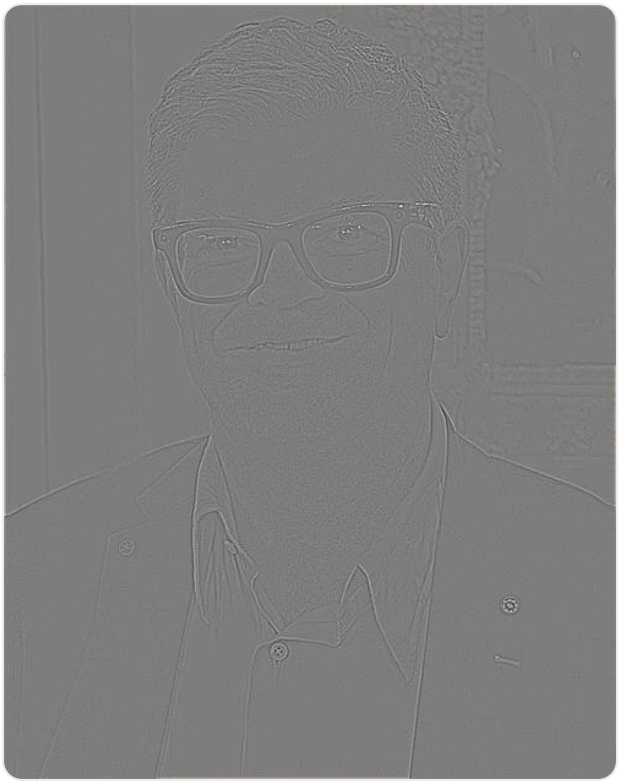
Sharpening the blurred image helps, but there are some details that were lost, like some wrinkles here and there in the shirt, etc. Sharpening can only exaggerate details that exist, and blurring removes some.



ORIGINAL (SHARP)



BLURRED



HIGH FREQUENCIES

Sharpening the original:



SHARPEN 1



SHARPEN 2



SHARPEN 3



SHARPEN 4



SHARPEN 5

Re-sharpening the blurred version:



RE-SHARPEN 1



RE-SHARPEN 2



RE-SHARPEN 3



RE-SHARPEN 4



RE-SHARPEN 5

2.2 Hybrid Images

Humans tend to see high frequencies up close but low frequencies from far away. This is the crux of the optical illusion. To do this we use the function to align the images; the low-frequency image is blurred and the high-frequency one is the impulse. For the low-frequency image, the energy captured by the FFT is more contained/constrained; for the high-pass FFT, the center is weaker, but everywhere else the other frequencies are kept. The hybrid combines both: we see Akhil in the center, while mine is on the outside. The sigmas are chosen by trial and error.

Full process · Aayan + Akhil (my favorite)

Aligned inputs (low-freq source / high-freq source):

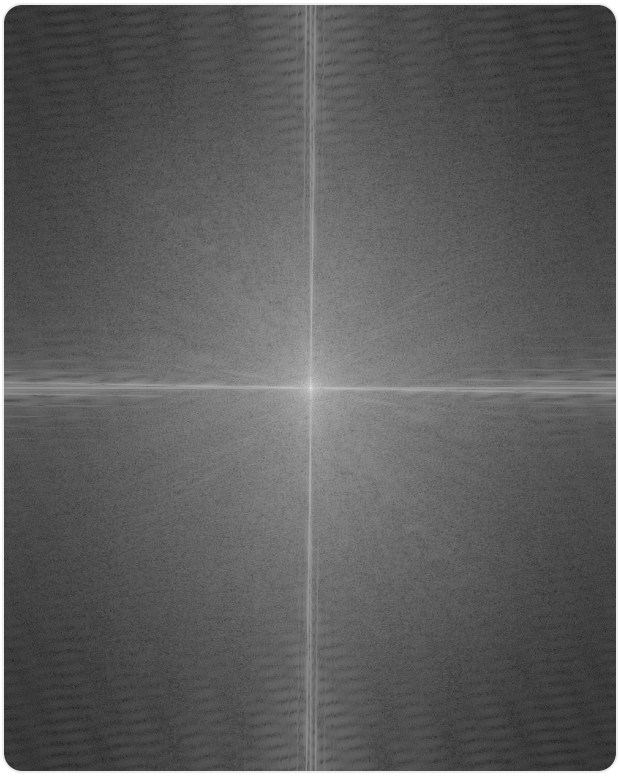


AKHIL · ALIGNED (LF SOURCE)

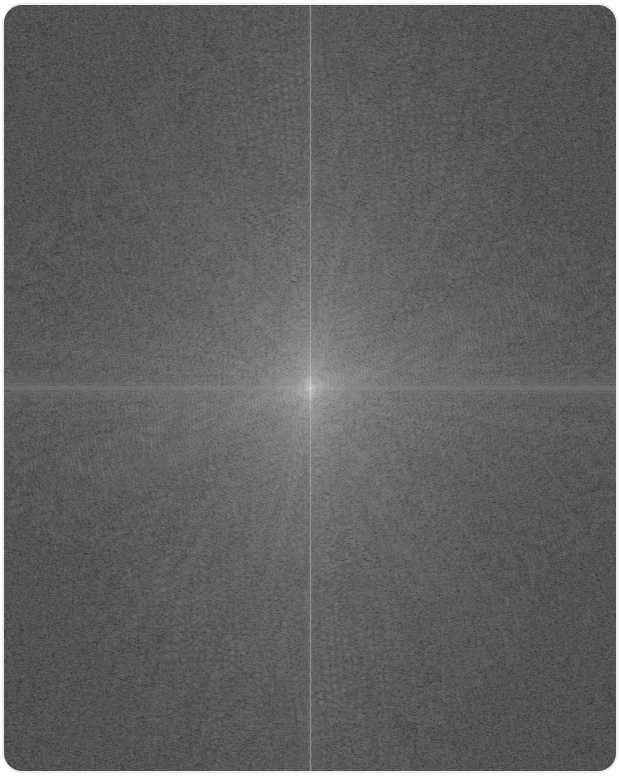


AAYAN · ALIGNED (HF SOURCE)

Log-magnitude FFTs of the aligned inputs:



FFT · AKHIL

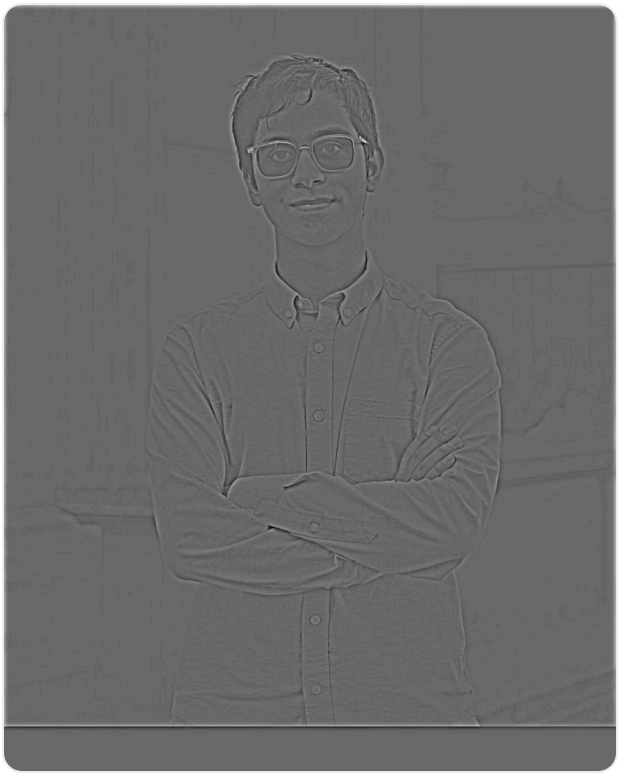


FFT · AAYAN

Filtered: low-pass / high-pass:

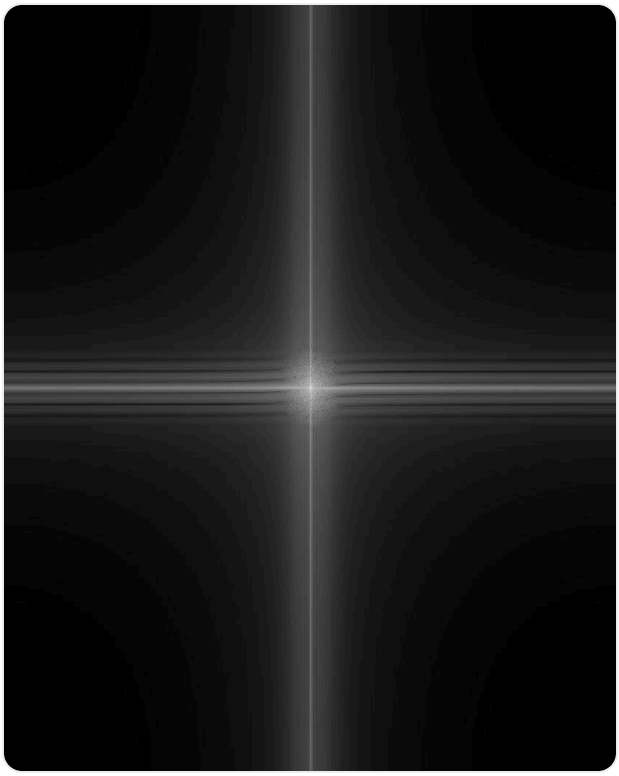


LOW-PASS (AKHIL)

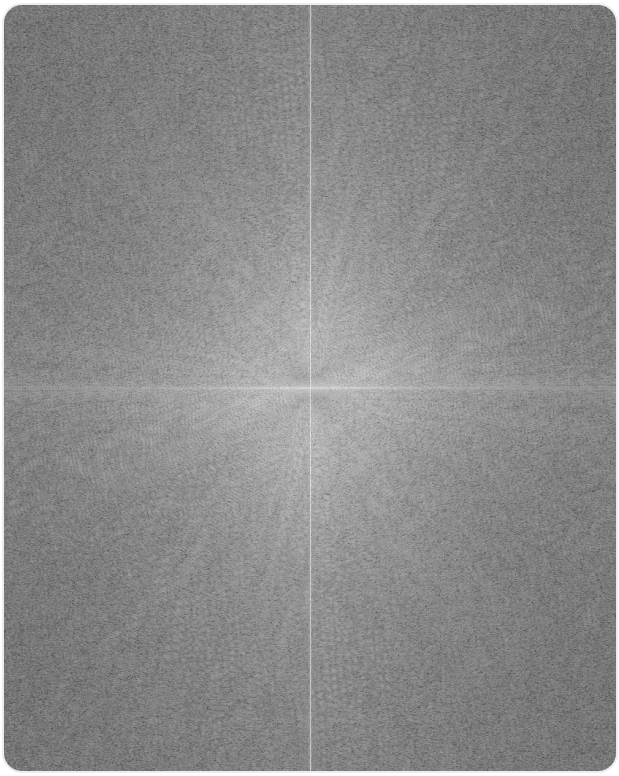


HIGH-PASS (AAYAN)

Log-magnitude FFTs of the filtered images:



FFT · LOW-PASS

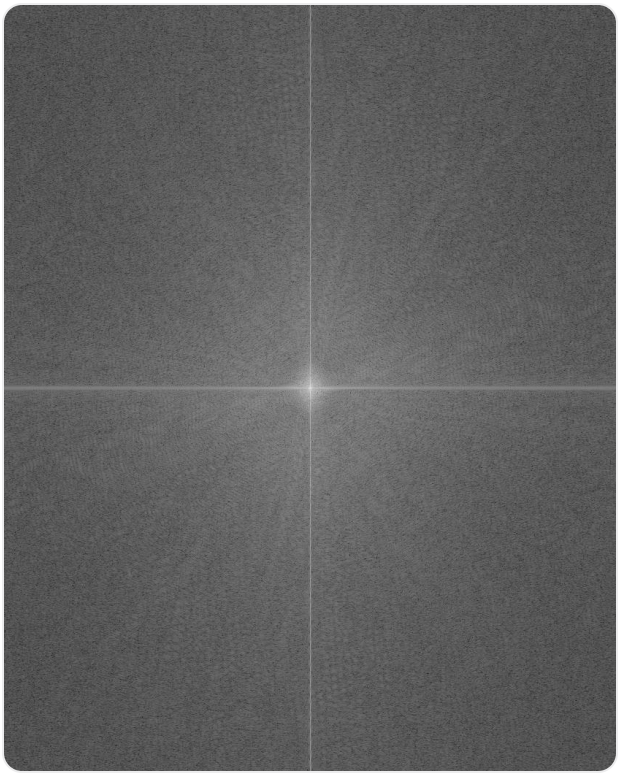


FFT · HIGH-PASS

The sigmas are chosen by trial and error.



HYBRID



HYBRID · FFT

Derek + Nutmeg



DEREK · ALIGNED (LF)



NUTMEG · ALIGNED (HF)



HYBRID

Bear + Oski





OSKI · ALIGNED (LF)

BEAR · ALIGNED (HF)



HYBRID

2.3 Gaussian & Laplacian Stacks

For the gaussian stack, level 0 is just the original image and each level after is the previous one blurred more, where I double the sigma each time (1, 2, 4, 8, 16). I don't downsample so everything stays the same size (this was specified in the spec). The laplacian stack is just the difference between two consecutive gaussian levels ($L_i = G_i - G_{i+1}$), so each one grabs a single level of detail. The early levels have the fine edges and texture, the middle ones have coarser stuff, and the last level is just the final gaussian which holds the low frequency color and lighting (very important to the image). This is intuitive when we see how we subtract (i.e at each level we subtract). If you add all the levels back up you get the original exactly. Since the laplacian levels have negative values too, I map zero to gray so you can see them, where brighter is positive and darker is negative.

Apple · Gaussian stack



LEVEL 0



LEVEL 1



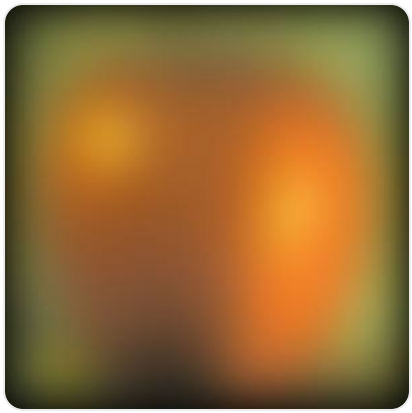
LEVEL 2



LEVEL 3



LEVEL 4



LEVEL 5

Apple · Laplacian stack



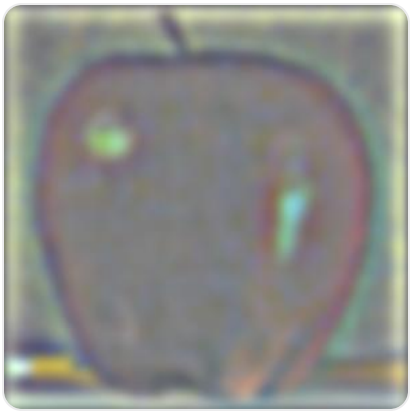
LEVEL 0

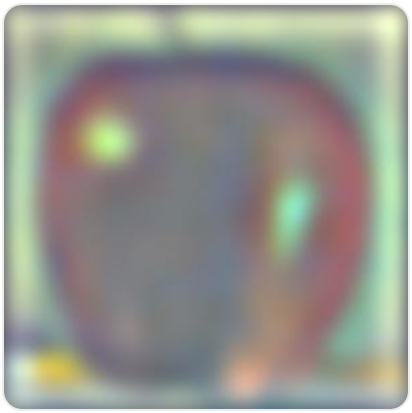


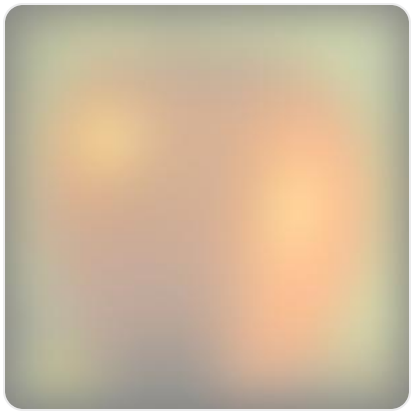
LEVEL 1



LEVEL 2







LEVEL 3 LEVEL 4 LEVEL 5

Orange · Gaussian stack





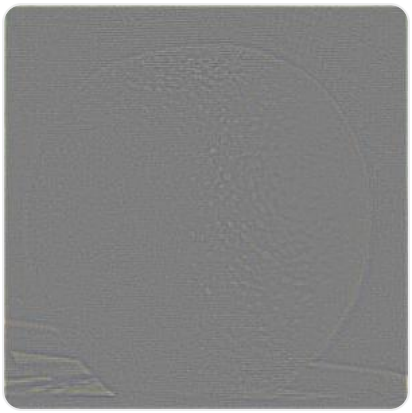
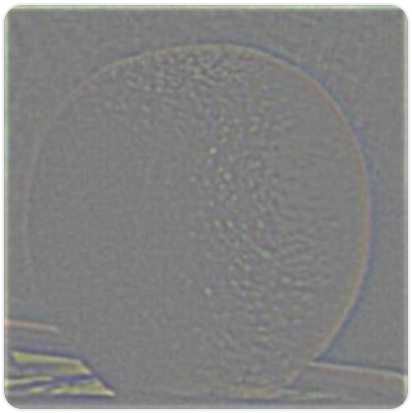
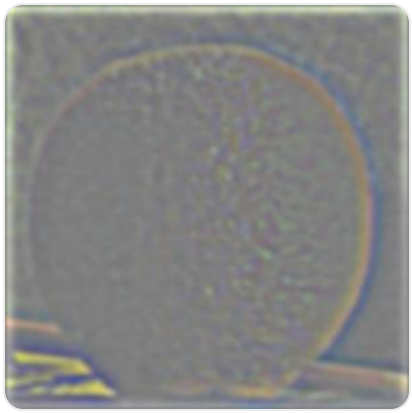
LEVEL 0 LEVEL 1 LEVEL 2



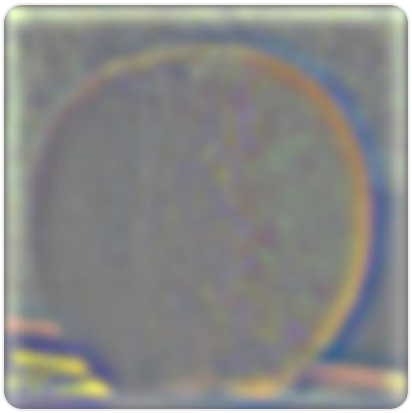


LEVEL 3 LEVEL 4 LEVEL 5

Orange · Laplacian stack

LEVEL 0 LEVEL 1 LEVEL 2



LEVEL 3



LEVEL 4

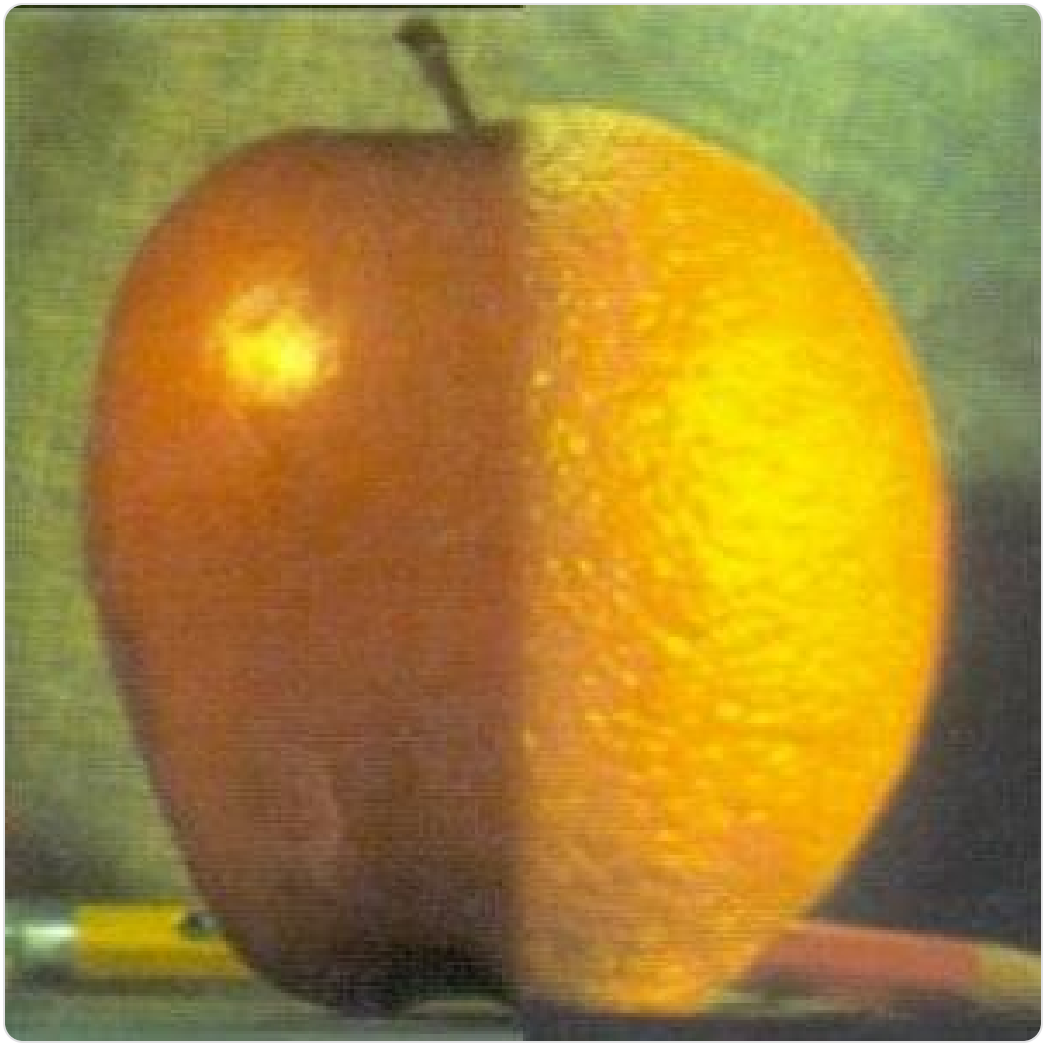


LEVEL 5

2.4 Multiresolution Blending (the Oraple)

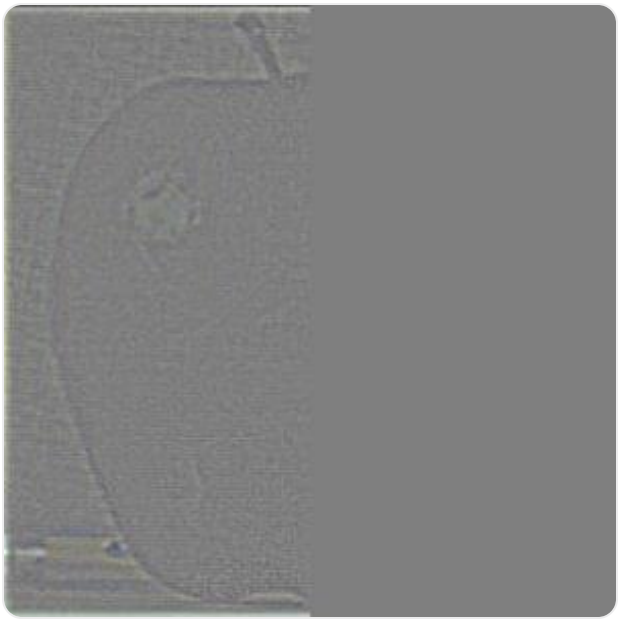
To blend I make a laplacian stack for each image and a gaussian stack for the mask, and at every level I combine them as $G_i(\text{mask}) \cdot L_i(A) + (1 - G_i(\text{mask})) \cdot L_i(B)$, then add all the levels up for the final image. This works because when the mask is sharper it corresponds to the levels that have the finer details, so those stay within their boundaries; but as the mask gets blurrier we get to the lower levels which (as mentioned before) correlate with stuff like lighting and base color, which makes those transition smoothly and gives a nice blend.

The Oraple



APPLE + ORANGE · VERTICAL SEAM

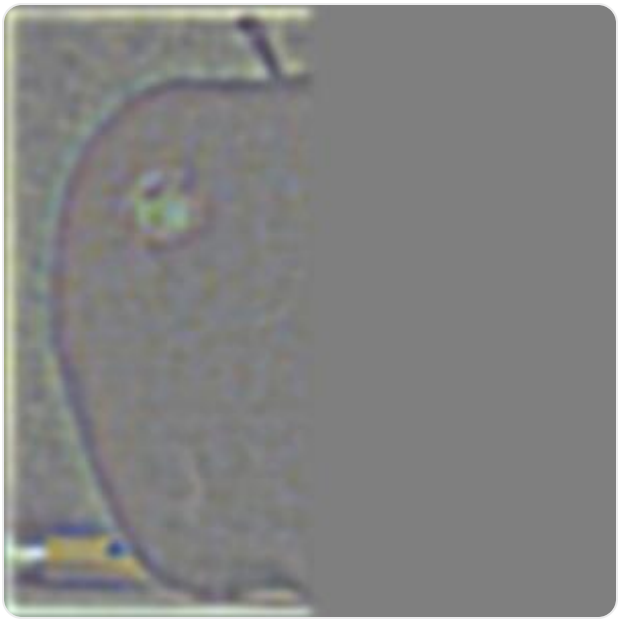
Recreation of Szeliski Figure 3.42 (masked Laplacians per level, combined):



APPLE 0



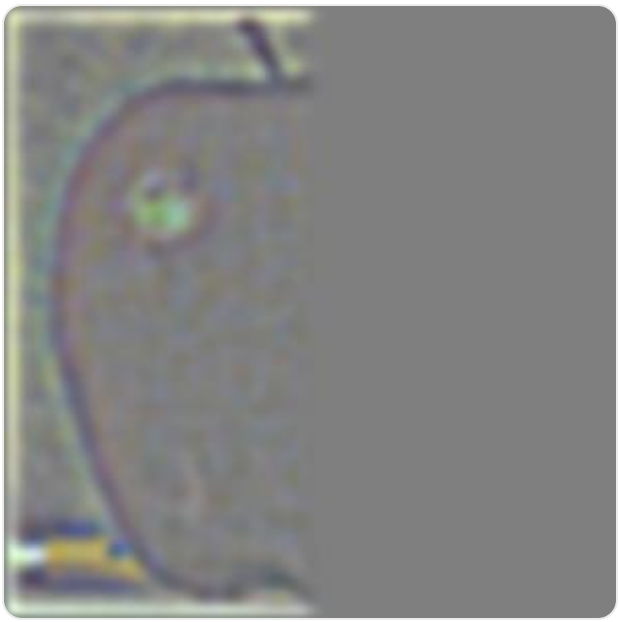
APPLE 1



APPLE 2



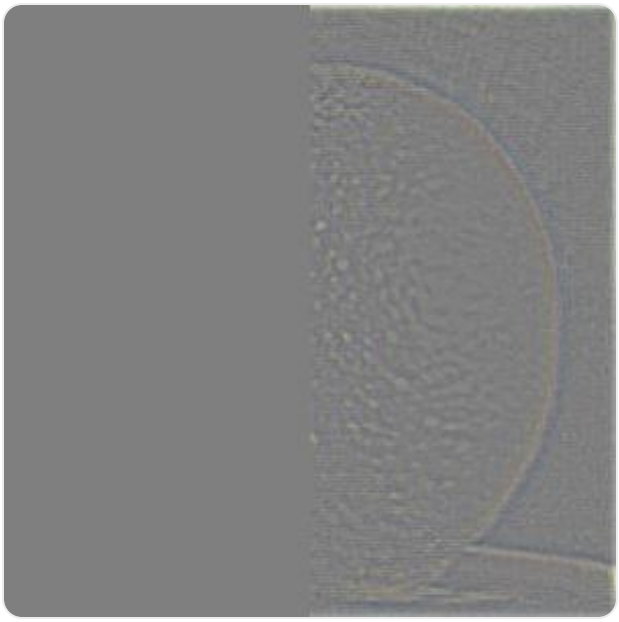
APPLE 3



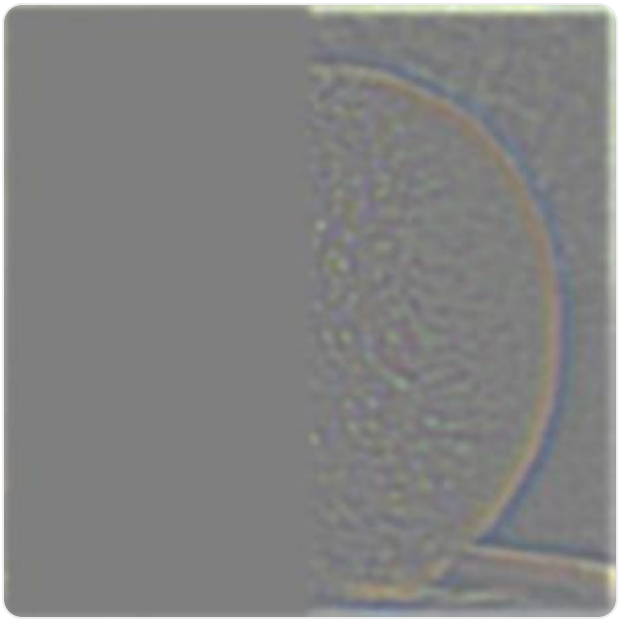
APPLE 4



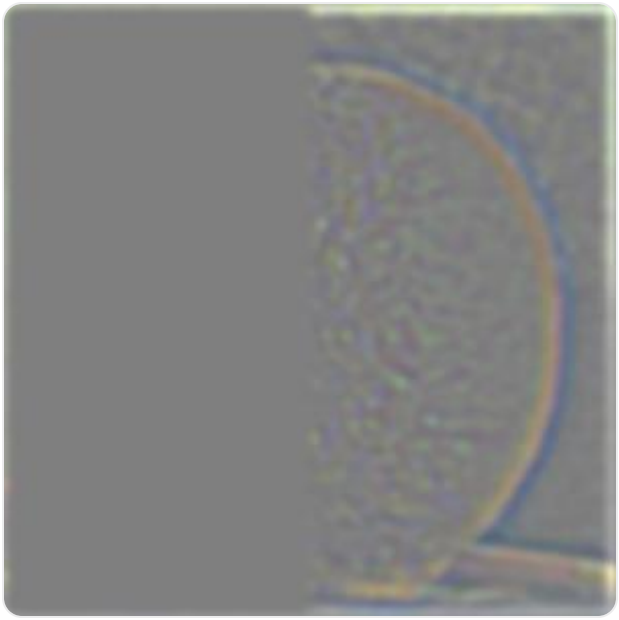
APPLE 5



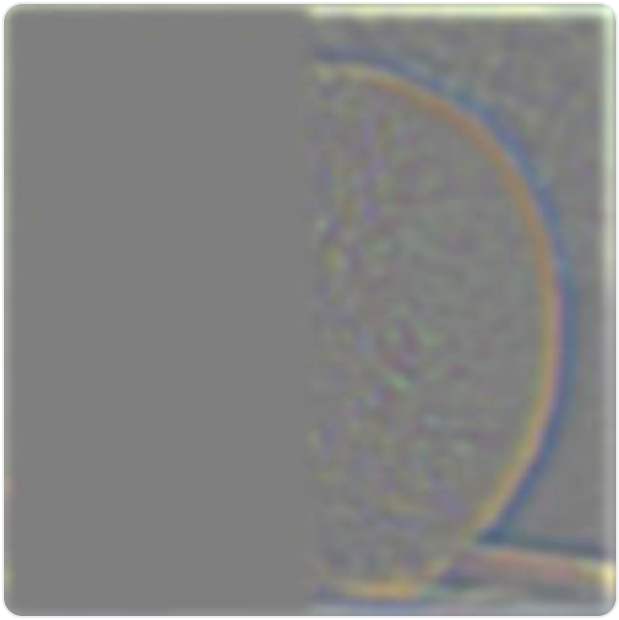
ORANGE 0



ORANGE 1



ORANGE 2



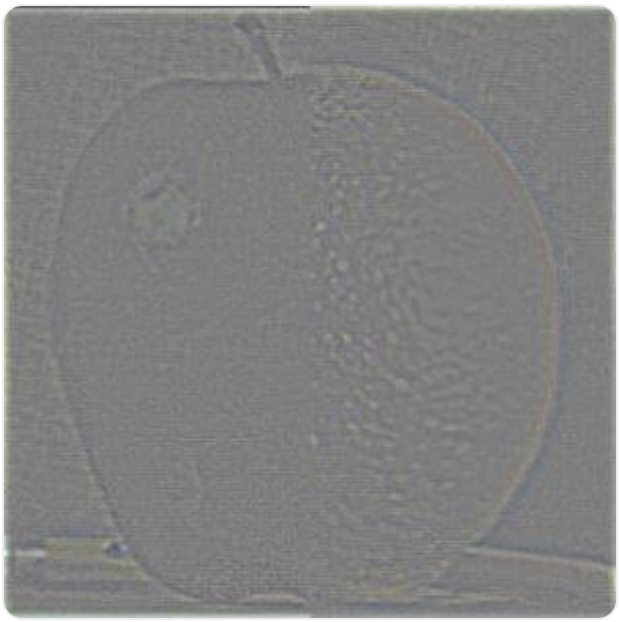
ORANGE 3



ORANGE 4



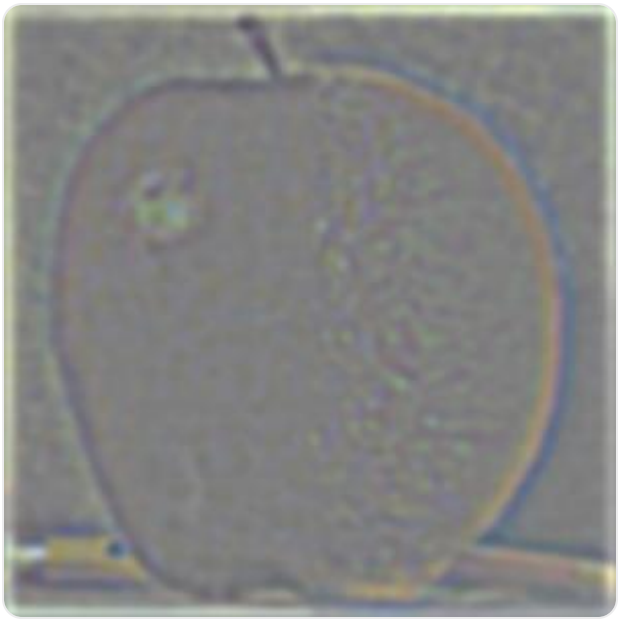
ORANGE 5



BLEND 0



BLEND 1



BLEND 2



BLEND 3

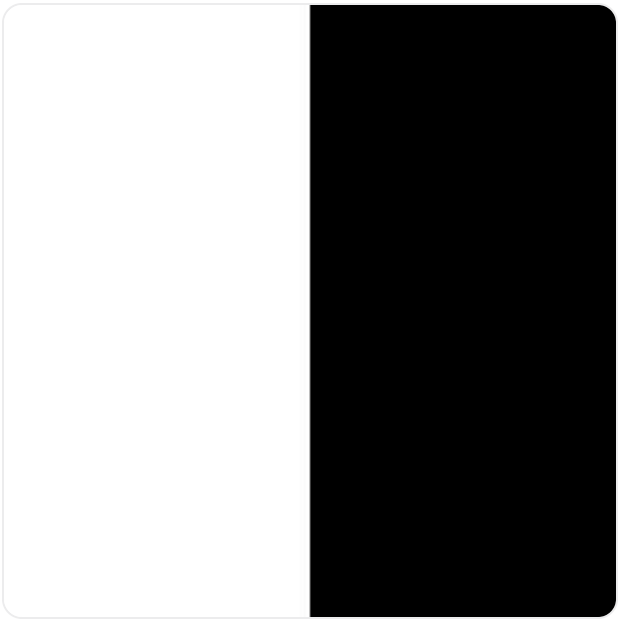


BLEND 4

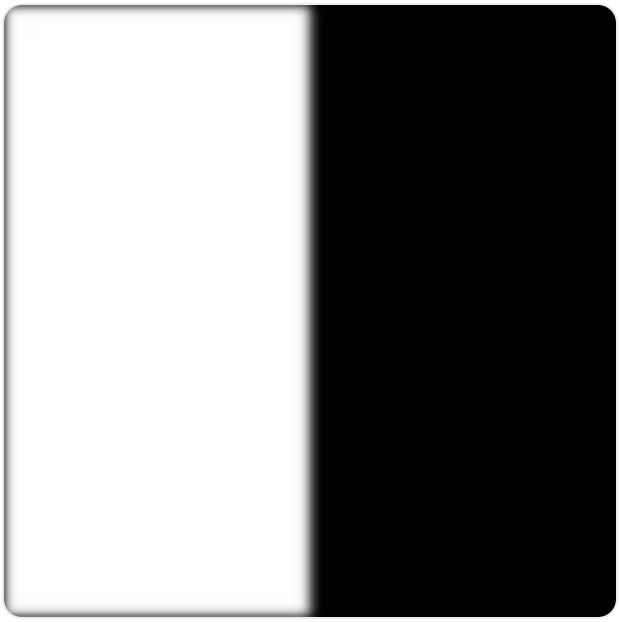


BLEND 5

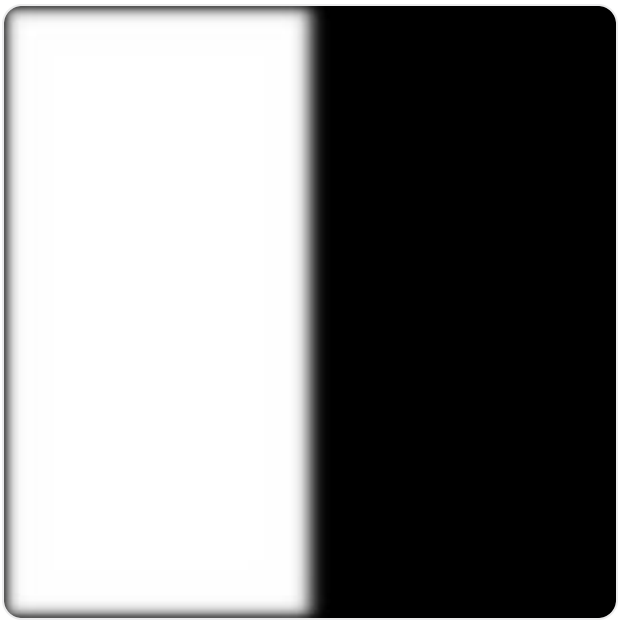
Blend process (masks, masked Laplacians, reconstructions):



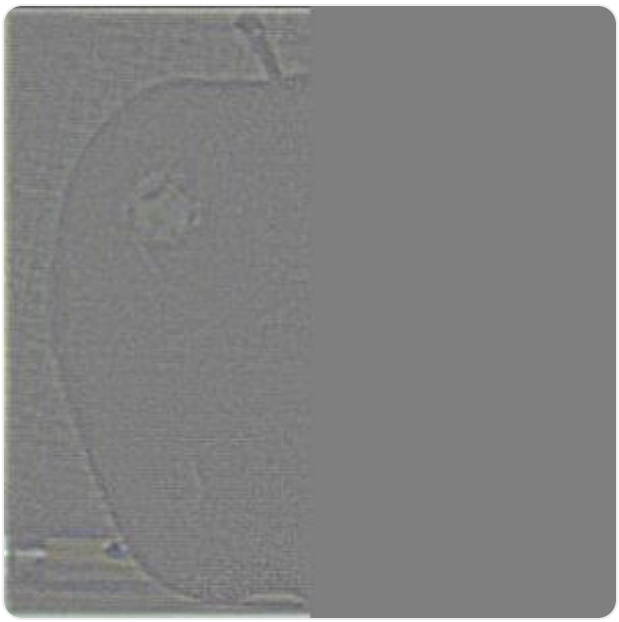
MASK · LVL 0



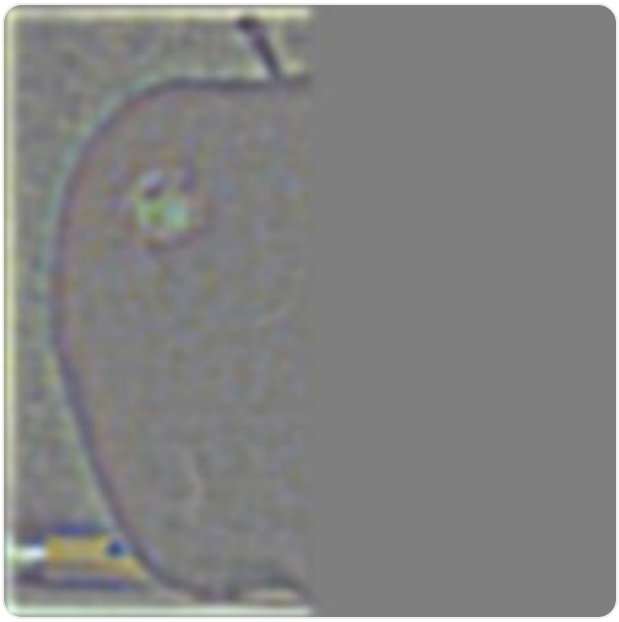
MASK · LVL 2



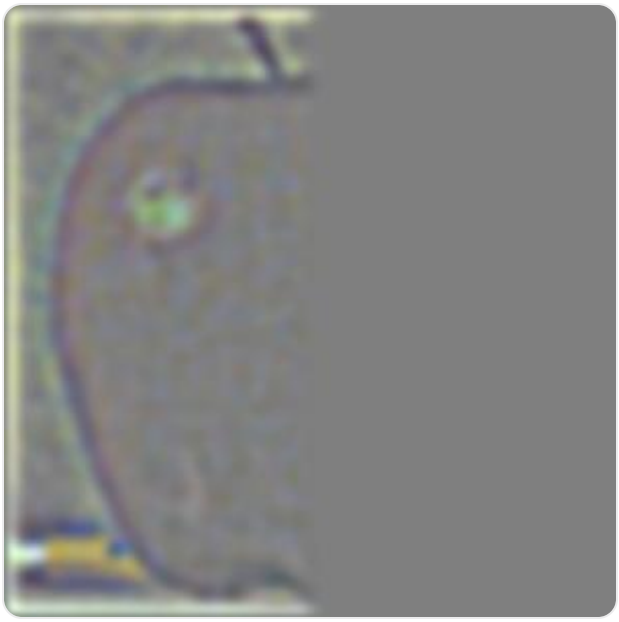
MASK · LVL 4



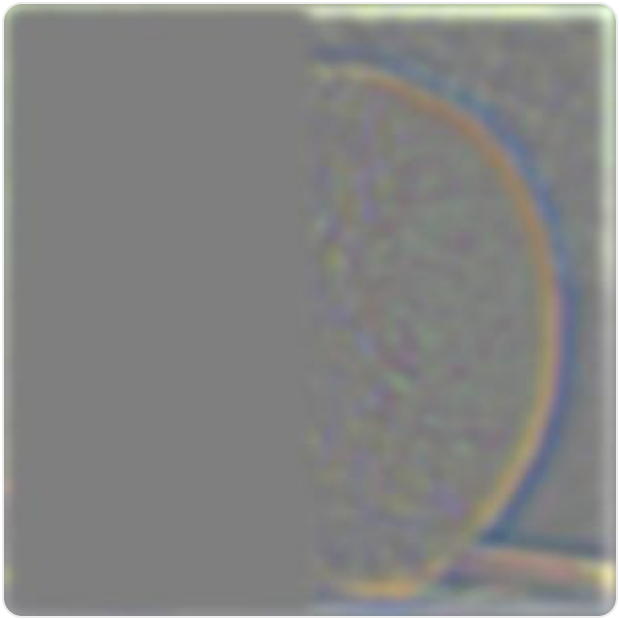
APPLE L · LVL 0



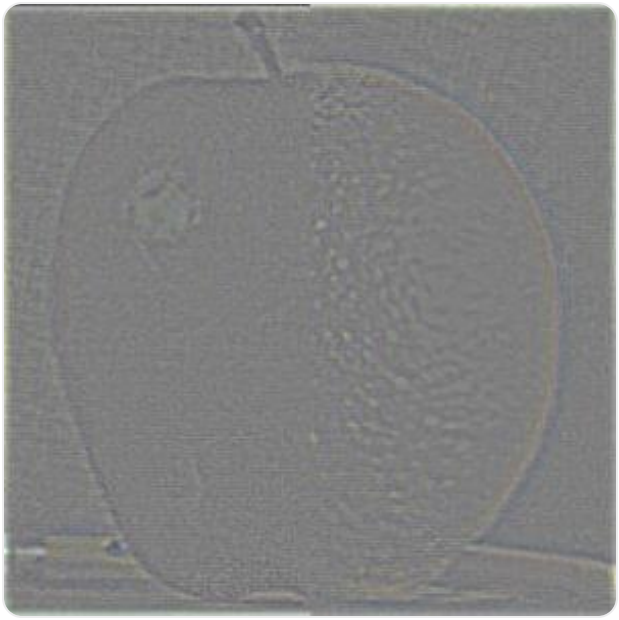
APPLE L · LVL 2



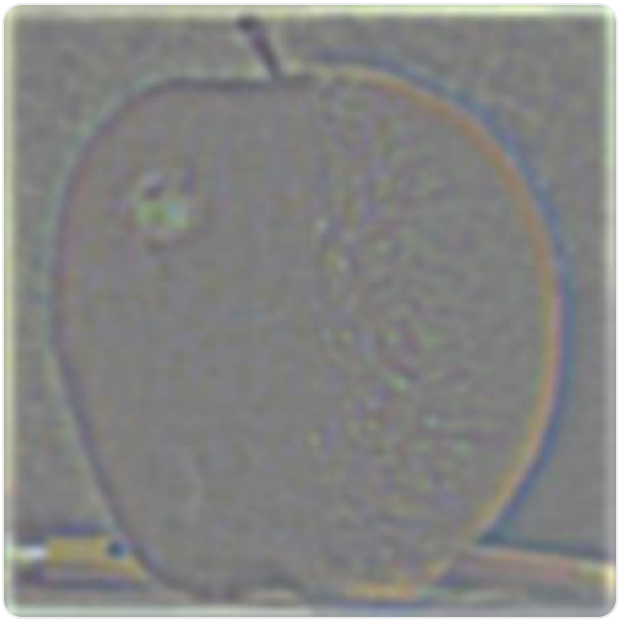
APPLE L · LVL 4



ORANGE (MASKED)



BLEND · LVL 0



BLEND · LVL 2



BLEND · LVL 4



RECONSTRUCTED APPLE



RECONSTRUCTED ORANGE

Here I show the gaussian mask and the masked apple and orange laplacian bands at levels 0, 2, and 4, their blended sums, and the reconstructed halves that add up to the oraple.

For my own blends, the earth and moon one uses a vertical mask just like the oraple: Earth is the left half and the Moon is the right half, and since both are the same size and lined up they merge into one planet that is half Earth and half Moon with a smooth seam down the middle. The sphere and campanile one uses an irregular dome-shaped mask (the outline

of the sphere), so the yellow sphere gets pulled into the campanile scene, and because the mask is blurred at the coarse levels it settles in instead of looking like a hard cutout.

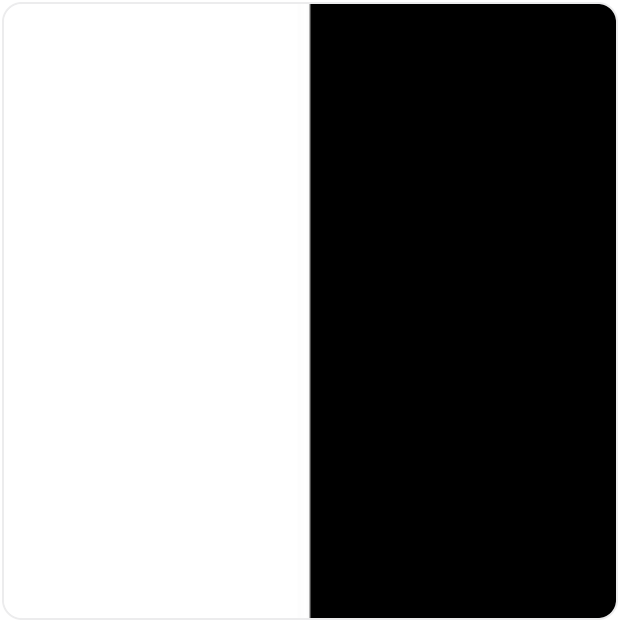
Custom blend 1 · Earth + Moon (vertical seam)



EARTH



MOON



MASK



BLENDED

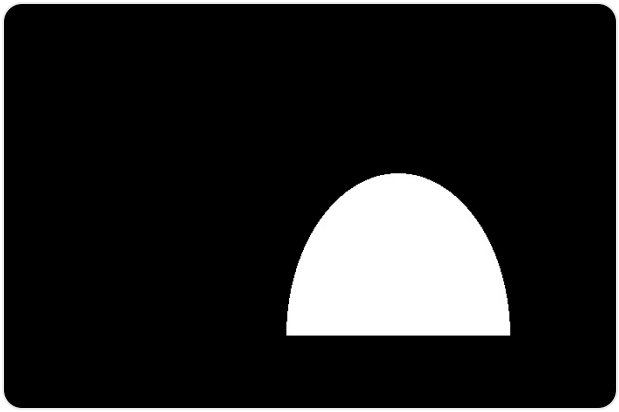
Custom blend 2 · Sphere + Campanile (irregular mask)



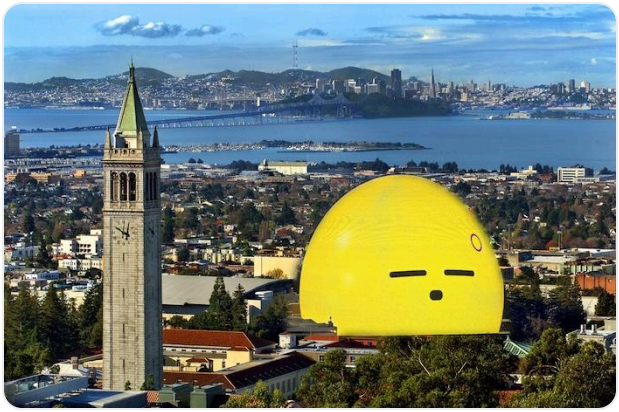
SPHERE



CAMPANILE



MASK



BLENDED

REFLECTION

Most Important Thing Learned

I learned how important (and relatively simple to work with) frequencies are. We saw that with gaussians we can make edge detection better. By also looking at the impulse we can add details to images (exaggerate them). This also unlocks seamless blending. The ability to look at and isolate these frequencies gives us so much control over an image. This

result is very cool to me, and I definitely want to play with using neural nets to do stuff with frequencies to get some generative properties.